Bangladesh University of Professionals

Faculty of Science and Technology
Department of Computer Science and Engineering

System Requirements Specification

ESP32 Based Smart Waste Bin

Real Time Monitoring and Collector Dashboard

Course information	Details
Course name	Computer Interfacing Laboratory
Course code	CSE 4102
Section	B
Group number	06
Date of submission	5 October 2026

Submitted by

Group member	Student ID
Sheikh Sifat Rahman	23524242108
Sajid Al Adeeb	23524242030
Abdullah Towhid	23524242054
Taqi Tahmid	23524242110

Table of Contents

1 Introduction 3

2 Existing System and Requirements Discovery 4

3 User and System Requirements 5

3.1 User Requirements 5

3.2 Functional Requirements 6

3.2 Functional Requirements Continued 7

3.3 Non Functional Requirements 8

4 Hardware and Software Requirements 9

4.1 Hardware Requirements and Budget 9

4.2 Software Requirements 10

5 System Design and Architecture 11

5.1 Circuit Diagram 11

5.2 Working Principle 12

5.3 Flowchart 13

6 System Interaction Model 14

7 Limitations and Future Scope 15

8 Appendix 16

8.1 Data and API Details 16

8.2 Test Plan 17

8.3 Settings and References 18

About this report

This report describes what the proposed Smart Dustbin should do and how its parts will work together. The project is still in the planning stage. The tests describe work to be carried out, rather than results already achieved.

Requirements use the word “shall” so that each expected behavior can be checked. Timing values and thresholds marked as proposed settings can be adjusted after testing.

1 Introduction

1.1 Background and motivation

A caretaker usually finds out whether a dustbin is full by walking up to it and looking inside. If the bin is nearly empty, the visit may not lead to any action. If it fills soon after the visit, waste may remain there until the next inspection. As the number of bins grows, keeping track of them becomes harder.

There is also a smaller problem at the bin itself. A person may need to touch the lid before throwing waste away. An automatic lid can make this easier, but it does not tell a collector when the bin needs to be emptied. This project brings these two needs together.

The Smart Dustbin will open when its IR sensor detects a hand. A separate ultrasonic sensor will measure the distance to the waste. The readings will then appear on a local dashboard, where a collector can see which bin needs attention and when it was last emptied.

1.2 Objectives

1. Open and close the lid without requiring the user to touch it.
2. Measure fill level using one ultrasonic sensor and show it as a percentage.
3. Read temperature and humidity and show useful information on an LCD.
4. Let collectors view bin locations, readings, opening counts, and collection history.
5. Estimate the next collection time and demonstrate the complete system on local Wi-Fi.

1.3 Scope

The project includes one physical bin, simulated bins, an ESP32, a local server, a database, and a browser dashboard. Both the hardware and the dashboard will use the same Wi-Fi network. Bin locations will be written as text addresses. The project does not include waste sorting, weighing, GPS maps, vehicle routes, or cloud access.

1.4 Intended users

People using the bin will interact with its automatic lid. Collectors will use the dashboard to plan visits and review collections. The project team will set up and test the system. Course evaluators will review how the hardware and software work together.

2 Existing System and Requirements Discovery

2.1 What existing systems offer

A normal dustbin is simple to use, but its condition is only known when someone checks it. Some small electronic projects improve this by adding an automatic lid. Arduino Education, for example, describes a trash can that uses an ultrasonic sensor and a servo to open when a person approaches [R4].

Arduino Cloud also describes a smart trash can with automatic opening and an option to open it through the cloud [R5]. These examples show how touchless access can work. Their cited descriptions focus on lid control rather than the collection records and local collector view needed for this project.

Table 2.1 Comparison of existing approaches

System	What it provides	What this project adds
Normal dustbin	A place to store waste; checked by hand.	Fill readings and a shared view of bin condition.
Arduino Education example [R4]	Proximity sensing and servo lid control.	Separate fill sensing, environment readings, and collection tracking.
Arduino Cloud example [R5]	Automatic opening and cloud control.	A collector dashboard designed to work locally without internet.

2.2 How the requirements were found

The starting point was the project proposal [R1]. The collector dashboard plan [R2] then explained how several bins would appear on one screen and how the readings would be stored. The course guideline [R3] provides the structure of this report. No interview or survey study is included.

2.3 The proposed solution

The updated design uses an IR obstacle sensor to detect a hand and one ultrasonic sensor to measure fill distance. The ESP32 handles the lid and sends readings through Wi-Fi. The local server turns those readings into fill information, collection records, and an estimate of the next collection time.

Node.js, Express, and SQLite replace the earlier proposal’s Python dashboard and USB data connection. Plain JavaScript keeps the browser interface simple. A manual collection button is also included, so the collector can record an emptying that automatic detection does not recognize.

3 User and System Requirements

3.1 User requirements

A person using the bin should not need to understand the electronics. A collector should be able to open the dashboard and quickly see what needs attention. The following requirements describe those needs.

UR01. The system shall open the lid when a hand is detected near the IR obstacle sensor.

UR02. The system shall close the lid after the hand has been removed for the configured time.

UR03. The system shall let a collector view all registered bins and open a separate page for each bin.

UR04. The system shall show a bin’s address, fill level, last collection time, and number of lid openings since collection.

UR05. The system shall show temperature, humidity, connection status, and the time of the latest reading.

UR06. The system shall show an estimated next collection time, or explain why there is not enough data to calculate it.

UR07. The system shall let the collector mark a bin as collected after emptying it.

UR08. The system shall continue local lid operation when Wi-Fi is unavailable and distinguish physical bins from simulated bins.

A typical visit

The collector opens the dashboard and sees that one bin is nearly full. After selecting it, the collector reads its address and checks when the data last arrived. The collector then empties the bin. A suitable drop in fill level records the collection automatically. If the drop is not recognized, the collector can use “Mark as collected.”

What the numbers mean

The usage count means lid openings, not items of waste. The collection time is a recorded event, not proof of a scheduled visit. The estimated next collection time is a guide based on recent fill changes. These meanings shall be clear on the dashboard.

3.2 Functional Requirements

At the physical bin

The process starts with a hand near the lid. Local control must respond first, even if the network is slow. Sensor readings can then be sent to the server without interrupting the person using the bin.

FR01. The ESP32 shall use the IR sensor for hand detection and the SG90 servo for lid movement. A hand that remains in front of the sensor shall not start repeated opening cycles.

FR02. The lid shall remain open while a hand is detected. After the area stays clear for a proposed 3 seconds, the lid shall close. A proposed 2 second cooldown shall prevent repeated triggers. Detection during closing shall request reopening.

FR03. The controller shall add one usage event when an opening movement completes. The servo angles shall be set for the actual lid. Since there is no position sensor, the count represents commanded opening cycles.

FR04. One ultrasonic sensor shall measure the distance to the waste. It shall not take fill samples while the lid is open or during the proposed 2 second settling period after closing. The device shall send the median of five usable samples, or null when a usable batch is unavailable.

FR05. The DHT11 shall provide temperature and humidity. A failed reading shall be sent as null rather than zero. The device shall send valid JSON values.

FR06. The ESP32 shall send readings and opening increments every 5 to 10 seconds during normal operation. Pending opening counts shall remain available for retry when a request fails.

FR07. The LCD shall show temperature, humidity, Wi-Fi state, and the latest fill value returned by the backend. It may rotate between screens. An old fill value shall be marked as old when it cannot be refreshed.

FR08. The controller shall keep checking the hand sensor during network failures. On restart, it shall initialize the hardware without recording a false opening event and attempt to reconnect.

To prevent a lost server reply from causing double counting, each outgoing batch shall have a session ID and sequence number. A retry shall use the same batch. New openings shall be kept separately until that batch is acknowledged. Appendix 8.1 explains this small addition to the original plan.

3.2 Functional Requirements Continued

At the server and dashboard

Once a reading reaches the server, it becomes part of the bin's record. The server checks the data before using it, so a failed sensor or a repeated request does not create misleading information.

FR09. The backend shall accept readings for registered bin IDs, check their types and ranges, and save them with server time. Invalid payloads shall return 400 and unknown bin IDs shall return 404. Duplicate batches shall be acknowledged without adding the opening count again.

FR10. The backend shall calculate fill percentage from each bin's calibrated empty and full distances. Invalid distance shall be excluded from fill calculations, while valid opening counts and environment readings shall still be processed.

FR11. The backend shall compare stable fill values to detect collection. The proposed rule uses three valid readings within 5 percentage points and a transition from at least 70% to at most 10%. A detected collection shall be saved once and reset the usage count.

FR12. The manual collection action shall ask for confirmation, save a collection event with method "manual," and reset the count. It shall clear the earlier detection window and establish a new fill baseline from fresh stable readings. It shall not create an assumed zero fill reading.

FR13. The backend shall estimate time to the proposed 90% target only when it has at least five stable points spanning ten minutes after collection and a positive fill rate. It shall show "Collect now" at the target and "Not enough data yet" when an estimate cannot be made.

FR14. The dashboard shall show a list of bins and a detail page for each bin. Details shall include location, fill, temperature, humidity, usage, collection history, estimated collection time, and a fill history chart.

FR15. The dashboard shall mark a bin offline after 30 seconds without an accepted reading. It shall identify old and missing readings clearly. Times shall be shown in Asia/Dhaka.

FR16. Simulated bins shall send readings through the same API as the physical bin. The dashboard shall label their source and shall not create fake sensor values itself.

FR17. The server shall save readings, counts, and collection changes together in a database transaction. Committed data shall remain available after a server restart.

3.3 Non Functional Requirements

A working system needs more than the correct features. The lid should respond promptly, the readings should be understandable, and a short network failure should not stop normal use. Table 3.1 gives proposed targets for the laboratory prototype. These have not yet been tested.

Table 3.1 Quality requirements

ID and quality	Requirement and way to check it
NFR01 Accuracy	At five known level heights, at least 9 of 10 settled readings at each height shall be within 5 percentage points of the expected calibrated fill. Uneven waste shall be tested separately.
NFR02 Response	Lid opening shall begin within 1 second of a valid hand detection. A newly posted reading shall appear on the dashboard within 15 seconds on a healthy local network.
NFR03 Reliability	A 30 minute run shall finish without a reset or server crash. A 60 second Wi-Fi outage shall not stop the lid. Retried batches shall not be counted twice.
NFR04 Safe movement	The lid shall use tested travel limits and a lightweight mechanism. Sustained hand detection shall prevent a close request. The prototype is not a certified system for preventing pinch injuries.
NFR05 Electrical safety	ESP32 inputs shall use compatible voltage levels. The ultrasonic ECHO shall use a voltage divider. The servo shall use a suitable 5 V supply with common ground, rather than the ESP32 3.3 V output.
NFR06 Power	The supply shall support measured peak demand during Wi-Fi use and lid movement. Twenty consecutive lid cycles shall not cause a brownout. Battery life is outside the current scope.
NFR07 Ease of use	A bin detail page shall open with one selection from the list. Fill colour shall be supported by a number or label. Missing readings shall never look like valid zero values.
NFR08 Local access	The complete dashboard, including chart files, shall work without internet once installed. It shall use the same local network as the hardware.
NFR09 Maintenance	Calibration values and thresholds shall be configurable. Sensing, calculation, storage, and display logic shall be kept separate for testing.
NFR10 Access control	The prototype shall run on a trusted demo network. Input checks shall be required. A shared device API key may be added, but the baseline does not provide encrypted traffic or collector accounts.

4 Hardware and Software Requirements

4.1 Hardware requirements and budget

Only one set of bin hardware is needed for the demonstration. The other bins will be simulated. Table 4.1 gives planning allowances in Bangladeshi taka. These are estimated budget amounts, not supplier quotations or confirmed purchase prices.

Table 4.1 Hardware and planned costs

Component and quantity	Purpose	BDT
ESP32 board ×1	Read sensors and connect through Wi-Fi	650
HC-SR04 ×1	Measure distance to waste	120
IR obstacle sensor ×1	Detect a hand near the lid	80
SG90 servo ×1	Open and close the lid	180
DHT11 module ×1	Read temperature and humidity	150
16x2 LCD with I2C backpack ×1	Show local information	300
Breadboard ×1	Hold the prototype circuit	120
Jumper wire set ×1	Connect the components	100
Resistors and capacitor set	Voltage divider and power support	70
Logic level shifter ×1	Connect a 5 V I2C LCD safely	80
Regulated 5 V supply ×1	Provide adequate current	300
USB cable ×1	Program and power the board as suitable	100
Bin, mounts, and linkage	Hold the components and move the lid	400
Subtotal	One physical prototype	2750
Extra allowance	10% for small additional parts	275
Total planned budget	Excludes existing laptop and Wi-Fi device	3025

The LCD is assumed to have an I2C backpack. This must be checked before final wiring. The servo must also be tested with the selected lid. A heavy lid may need a different motor or a lighter mechanism.

4.2 Software Requirements

The hardware needs firmware to read sensors and move the lid. The computer then needs a small server to store the readings and a browser page to show them. Using these three parts keeps the work easy to build and test in stages.

Table 4.2 Software requirements

Part	Software	Purpose
Firmware	Arduino IDE, ESP32 board support, C/C++	Build and upload the ESP32 program.
Device communication	ESP32 Wi-Fi and HTTP client support	Send readings to the local server.
Hardware libraries	ESP32 compatible servo, DHT, and I2C LCD libraries	Control the selected modules. Exact library versions will be fixed during implementation.
Backend	Node.js and Express	Check data, calculate values, and serve the dashboard.
Database	SQLite with better-sqlite3 proposed	Store bin details, readings, and collection events.
Dashboard	HTML, CSS, and plain JavaScript	Show the collector interface in a browser.
Charts	Chart.js bundled locally	Draw fill history without an internet download.
Simulator	Node.js script	Send test readings using the real API.

Local network setup

The ESP32, server computer, and dashboard device will join the same local network. The network must allow devices to reach each other. A tested hotspot can be used if a university network blocks this traffic. The selected ESP32 requires a compatible 2.4 GHz Wi-Fi network [R6].

The proposed server port is 3000. The computer should have a fixed local address or a DHCP reservation so the ESP32 can find it again. The firewall must allow the server’s local traffic. Software packages and chart files will be installed before the demonstration.

Time and storage

The server will record time in UTC and the dashboard will display it in Asia/Dhaka. The computer clock should be checked before the demo. Raw readings may be kept for seven days, while collection records remain stored. If old data is removed, an estimate must still meet the minimum data requirement before it is shown.

5 System Design and Architecture

5.1 Proposed circuit diagram

Figure 5.1 shows the proposed signal connections. The exact board and module voltages must be checked before assembly. GPIO 23 is used for TRIG so the design does not rely on GPIO 5, which has a boot configuration role. All components share ground.

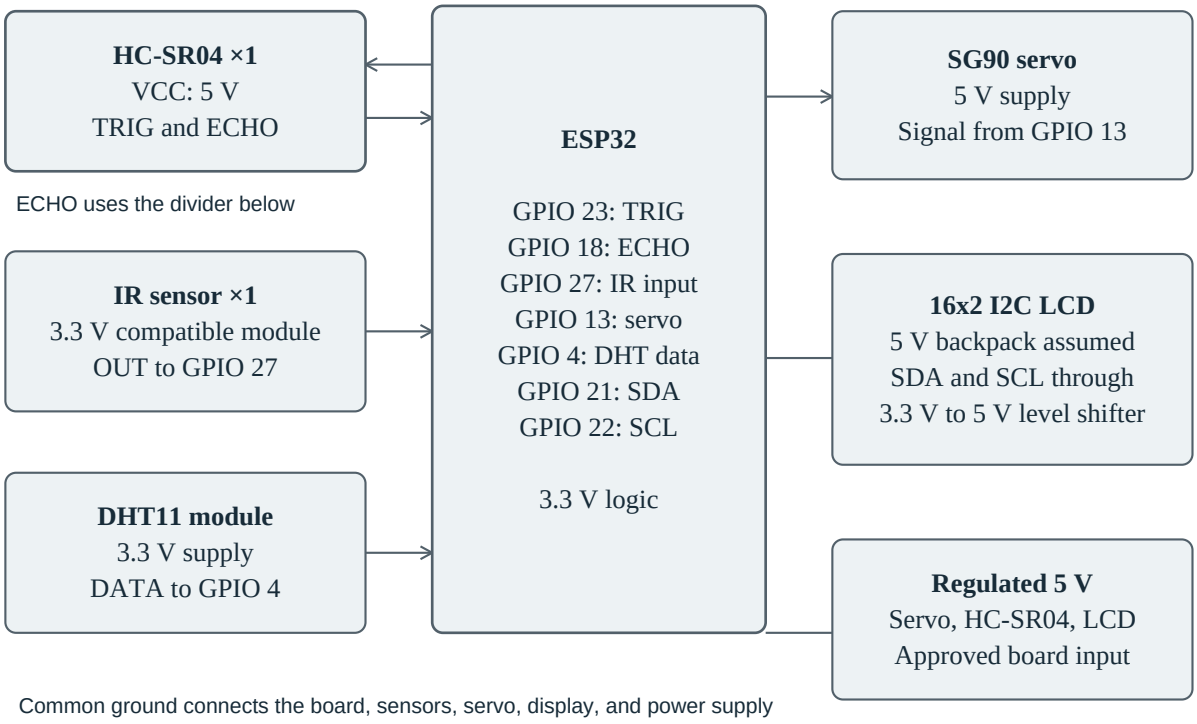

Figure 5.1 Proposed wiring for the physical bin.

Protecting the ultrasonic input

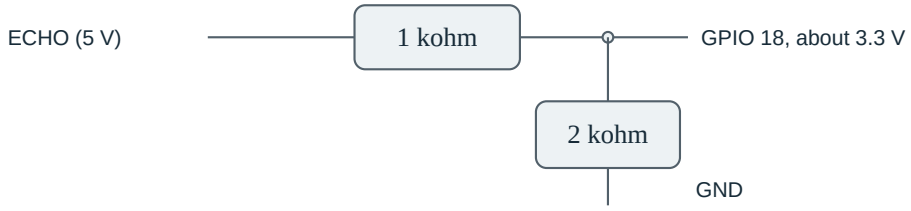

Figure 5.2 ECHO passes through a 1 kilohm resistor to the GPIO node. A 2 kilohm resistor connects that node to ground. The nominal output is 3.33 V.

The servo must not draw power from the ESP32 3.3 V pin. Use a suitable 5 V supply and common ground. A capacitor of 100 to 470 microfarads near the servo may help with short current peaks. A 5 V LCD backpack needs a suitable I2C level shifter. Check the IR output level and DHT pull-up before connection. Follow the selected board’s power guidance when USB and external power are both present [R6].

5.2 Working Principle

The system begins at the bin but finishes at the collector’s screen. Figure 5.3 shows the path between them. Physical and simulated readings enter the same server, which keeps the dashboard behavior consistent.

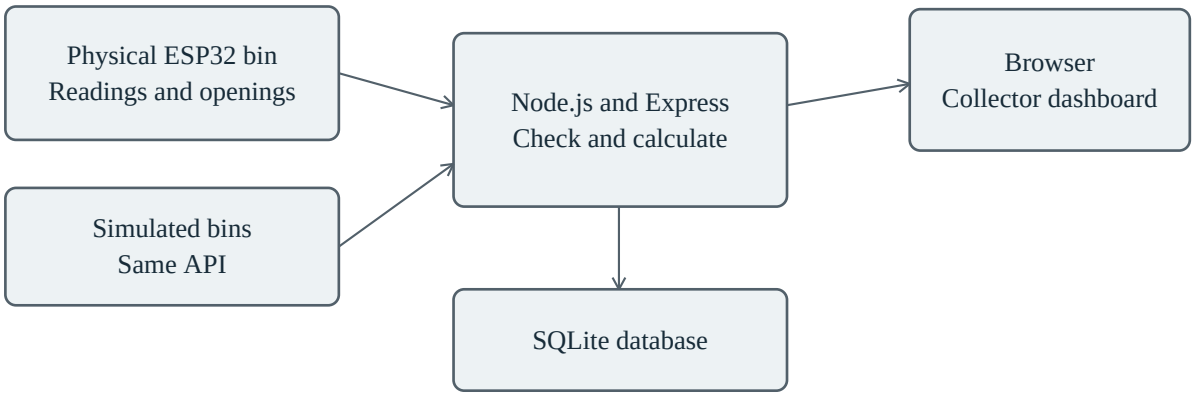

Figure 5.3 The local path from readings to the dashboard.

Turning distance into fill

The empty distance E is measured from the sensor to the empty bin floor. The full distance F is measured to the highest permitted waste level. For a usable reading d, the server calculates:
Fill percentage = 100 × (E minus d) / (E minus F)
The result is limited to 0 through 100. Calibration must satisfy E > F > 0. For example, E = 40 cm, F = 5 cm, and d = 19 cm gives 60% fill.

Recognizing a collection

A single low reading may be noise. The server therefore waits for three readings whose spread is no more than 5 percentage points and takes their median. If the previous stable fill was at least 70% and the new stable fill is at most 10%, it records a collection. It will not keep recording collections while the bin stays empty.

The manual button covers an emptying that does not meet this rule. After confirmation, it records the event and waits for fresh stable fill to set the new starting level. Earlier stable readings are cleared so they do not trigger another collection for the same emptying.

Estimating the next visit

After collection, the server divides the rise in stable fill by the elapsed hours. With enough data and a positive rate, it divides the remaining fill to the target by that rate. The result is an estimate, not a promise. The dashboard shows “Collect now” at 90%, or explains when data is missing, too old, or insufficient.

5.3 Flowchart

The lid and the data connection have different timing needs. The lid must respond while a person is using it. Data can be retried later. These two loops therefore run without one blocking the other.

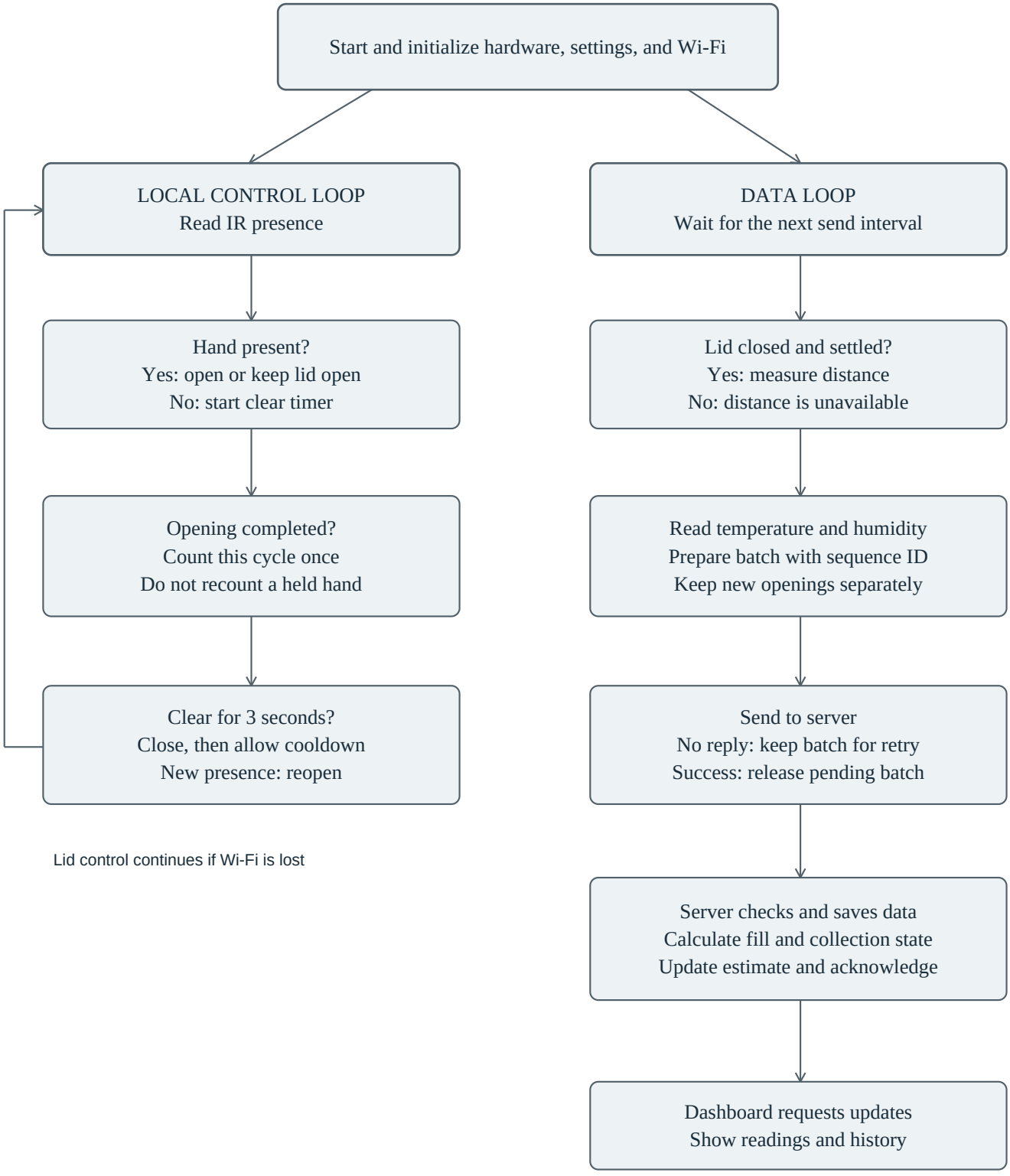

Figure 5.4 Main flow of the proposed system. Timings are starting values for testing. The LCD updates from local readings and the latest fill returned by the server.

6 System Interaction Model

6.1 Use case diagram

The bin user interacts with the lid. The collector works with the dashboard. The project operator handles setup and calibration. Figure 6.1 keeps these responsibilities separate.

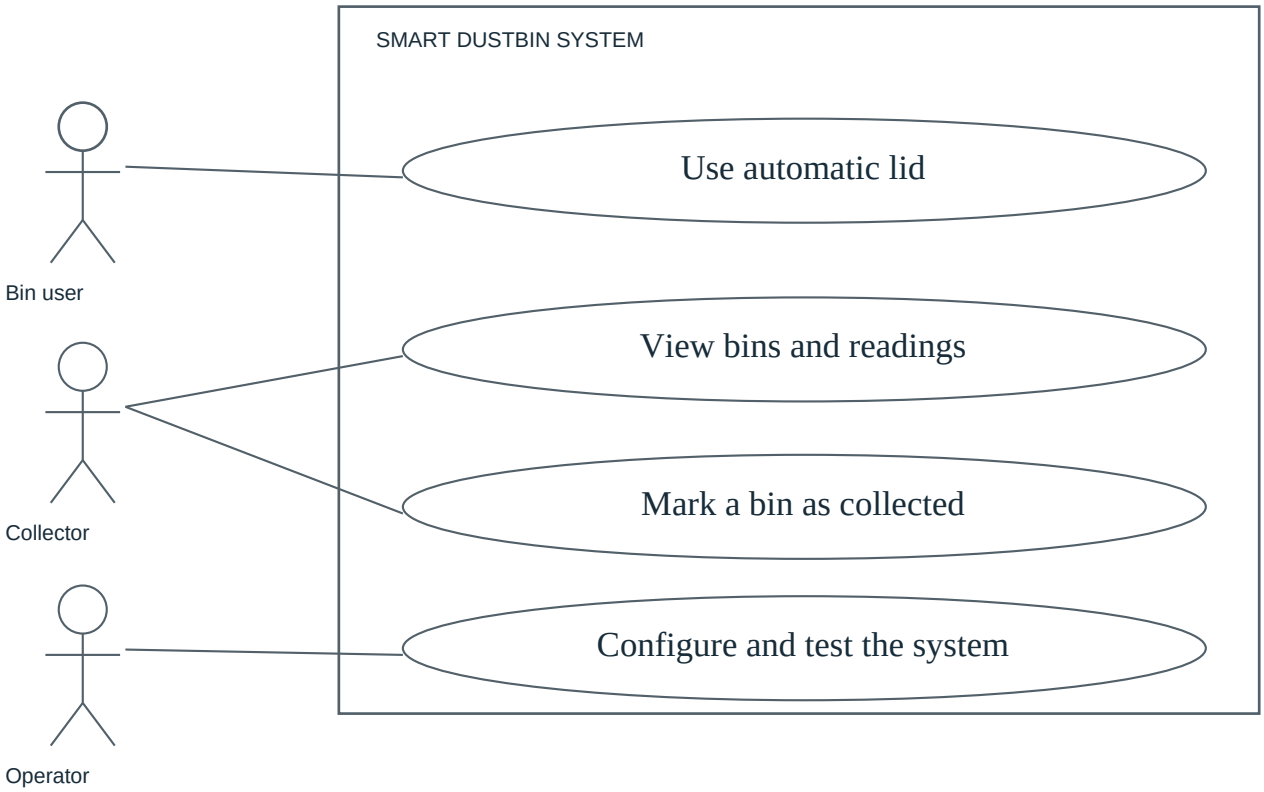

Figure 6.1 Main interactions with the Smart Dustbin.

6.2 Use the automatic lid

The bin is powered and ready. A user brings a hand near the sensor, and the controller opens the lid. The user drops waste and removes the hand. After the clear delay, the lid closes. One opening is counted. If Wi-Fi is lost during this process, local operation continues.

6.3 Inspect and collect a bin

The collector opens the bin list, selects a bin, and checks its fill and last update time. After emptying it, the collector waits for automatic detection. If needed, “Mark as collected” opens a confirmation that names the selected bin. Confirming saves a manual event and resets usage. Cancelling leaves the record unchanged.

If the server cannot be reached, the page shall show that the action failed and shall not display a successful collection. While saving, the button shall be disabled. A unique action ID shall make a retry safe if a server reply is lost.

7 Limitations and Future Scope

7.1 What the prototype cannot prove

The demonstration will use one real bin. Simulated bins will help show how the dashboard handles several records, but they cannot prove that many physical bins would perform equally well outside the laboratory.

Fill sensing also has limits. A level surface reflects sound differently from a loose pile of paper or an angled object. The measured percentage describes relative fill height, which may differ from the actual volume of waste. Calibration and repeated samples can help, but they cannot remove every error.

Opening counts are another estimate of activity. A person may open the lid without dropping anything, or place several items inside during one opening. Pending counts may also be lost if the ESP32 loses power before sending them. There is no separate sensor to confirm that the lid physically reached its intended position.

Automatic collection detection works only for the specified change in stable fill. It may miss partial emptying, early collection, or a gradual drop through several intermediate levels. The manual collection button covers these cases, but it relies on the collector entering the event correctly.

The next collection estimate assumes that recent filling continues at a similar rate. Real use can change during the day. A fast demonstration may also produce an estimate of only a few minutes. The system will label this as an estimate and avoid displaying one when there is too little reliable data.

The DHT11 provides supporting information, not a certified heat or fire warning. The circuit is not designed for exposure to water or heavy waste. The lid has no dedicated obstruction sensor. Finally, the local dashboard depends on a running computer and a trusted Wi-Fi network.

7.2 How the project could grow

The first useful step would be to test more physical bins for a longer period. This would show how often fill readings drift and how well the collection rules work with real waste. The findings could guide better sensors, revised mounting, and more suitable thresholds.

Later versions could preserve counts across power failures, add lid position or obstruction sensing, and improve the enclosure. User accounts, encrypted communication, and backups would be needed before wider use. Maps, routes, and remote access could then be considered as separate additions.

8 Appendix

8.1 Data and API details

Each message identifies a registered bin and contains its current readings. A null sensor value means that the reading is unavailable. It does not mean the measured value is zero.

Table 8.1 Reading fields

Field	Meaning and rule
binId	Registered bin identifier.
sessionId and sequence	A device session identifier and a nonnegative batch number. Together with binId, they identify a batch so retries are counted once. A reboot creates a new session.
distanceCm	Positive usable distance in centimetres, or null.
tempC and humidity	Temperature and relative humidity, or null. Values must be finite and within the selected sensor’s accepted range; humidity is limited to 0 through 100.
opensDelta	Nonnegative integer giving the openings in this batch.

Table 8.2 API endpoints

Method and path	Purpose
POST /api/readings	Accept sensor data. Return 200 after saving or recognizing a duplicate, 400 for malformed data, and 404 for an unknown bin.
GET /api/bins	Return the bin list and latest status.
GET /api/bins/:id	Return details, freshness, history summary, and estimate or reason. The ESP32 may use this to refresh LCD fill.
GET /api/bins/:id/readings	Return a limited set of recent readings for charts.
GET /api/bins/:id/collections	Return collection history.
POST /api/bins/:id/collect	Save a confirmed manual collection. A unique action ID makes repeated submission safe.

Stored records and event order

The database stores bins, readings, and collections. Each bin has its address, source type, calibration, settings, usage, and latest state. Readings include time and validity. Collections include time and whether detection was automatic or manual.

For each accepted batch, the server first checks for duplicates and adds its openings. It then evaluates fill and collection. A detected collection resets the count in the same transaction. Openings within the detection delay may therefore be included in that reset. After manual collection, fresh stable readings establish the baseline before automatic detection resumes.

8.2 Test Plan

These tests will be carried out during development and integration. For each one, the team will record the inputs, observed result, and whether the requirement was met. No completed test result is claimed here.

Table 8.3 Planned acceptance tests

Test and coverage	Test action	Expected result
T01: FR01 to FR03	Hold a hand near the sensor for 10 seconds, then remove it.	One opening count. Lid stays open, then closes after the clear delay.
T02: FR04, FR10, NFR01	Take ten samples at each of five known level heights.	At least 9 of 10 at each height are within 5 percentage points of expected fill.
T03: FR05, FR09, FR10	Send null sensor values with an opening increment of two.	No false zero values. Two openings counted; invalid distance excluded.
T04: FR11	Send stable 75% readings followed by stable 8% readings.	One collection recorded and usage reset. Further low readings do not repeat it.
T05: FR11	Send one low outlier between high readings.	No collection from the single outlier.
T06: FR06, FR09, FR17	Lose a server reply after saving, then resend the same batch.	The batch is acknowledged without adding openings twice.
T07: FR08, FR15	Disconnect Wi-Fi for 60 seconds while using the lid.	Lid still works. Offline status appears after the timeout and next dashboard refresh.
T08: FR12	Confirm a manual collection, retry the same action, then send fresh low readings.	One manual event. Count resets once. No extra automatic event from earlier high readings.
T09: FR13	Try too little data, a flat rate, a rising rate, and 90% fill.	Reason shown when unavailable, estimate for suitable data, and Collect now at target.
T10: FR09, FR16, FR17	Send bad data; run physical and simulated bins; restart server.	Bad data rejected. Bins remain separate. Saved records survive restart.
T11: NFR02 to NFR06	Measure response and current; run for 30 minutes and 20 lid cycles.	Timing targets met. No crash or brownout. Travel and logic levels remain suitable.
T12: FR07, FR14, NFR07, NFR08	Remove internet but retain Wi-Fi; inspect all pages and LCD.	Local dashboard and charts work. Old values are labelled. Bin details are easy to reach.

8.3 Settings and References

Starting settings

The values below are proposed starting points. The team will adjust them after calibration and keep the settings consistent between the server, firmware, and report. The exact LCD module and compatible libraries still need to be selected.

Table 8.4 Proposed starting settings

Setting	Proposed value
Reading interval and page refresh	5 to 10 seconds and 5 seconds
Clear delay and cooldown	3 seconds and 2 seconds
Stable fill window	Three valid values within 5 percentage points
Collection transition	At least 70% down to at most 10%
Collection target and offline time	90% and 30 seconds
Estimate readiness	Five stable points spanning at least ten minutes
Fill colours	Green below 50%; yellow from 50% to below 80%; red at 80% or more. The collection target is separate.

Before the first full demonstration

Measure the empty and full distances. Check IR sensitivity and output voltage. Set servo travel without forcing the lid. Confirm safe power and signal connections. Test level heights, automatic collection, manual collection, and network recovery. Check the computer clock and clearly label simulated bins.

References

[R1] Group 06. Smart Dustbin Project Proposal Corrected. Computer Interfacing Laboratory, Bangladesh University of Professionals. Submitted 21 September 2026. Supplied project proposal.

[R2] Smart Dustbin: Collector Dashboard, Planning Document. Supplied planning text. Includes confirmed choices, suggested settings, and open decisions.

[R3] System Requirements Specification Report Guideline. Supplied course guideline. Defines the report structure and page limit.

[R4] Arduino Education. Trash Can Project. Arduino Education project page. Accessed 4 October 2026.

[R5] Arduino Cloud. Smart Trash Can. Arduino Cloud project page. Accessed 4 October 2026.

[R6] Espressif Systems. ESP32 Series Datasheet. Official ESP32 datasheet. Accessed 4 October 2026. Check the selected development board separately.